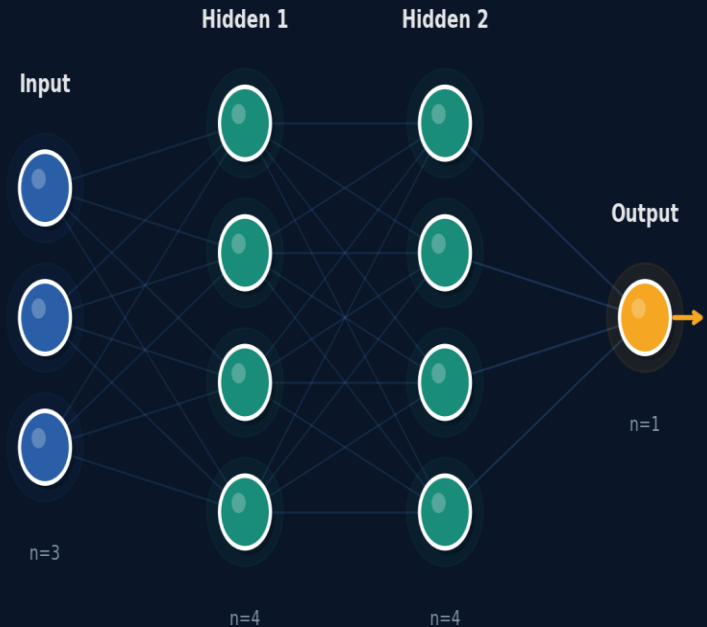


Neural Networks

Từ Neuron đến Backpropagation

Học Máy Nâng Cao
TS. Cao Tiến Dũng



SECTION A

Warm-up & Động Lực

Tại sao cần Neural Network?

Tại sao cần Neural Network?

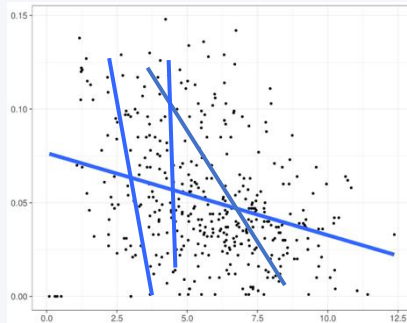
Giới hạn của mô hình tuyến tính

❌ Giới hạn của Logistic Regression

- Chỉ học được 1 đường phân tách tuyến tính
- Bài toán XOR: 4 điểm, không đường thẳng nào tách được
- Thất bại với dữ liệu phi tuyến thực tế

Bảng XOR:

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0



💡 Ý tưởng đột phá

- 1 LR không đủ → dùng nhiều LR cùng lúc
- Mỗi LR học 1 "góc nhìn" khác nhau về dữ liệu
- Kết hợp nhiều góc nhìn → hiểu cấu trúc phức tạp hơn
- Mỗi neuron = 1 bộ Logistic Regression thu nhỏ

💡 Neural Network = ý tưởng đơn giản: dùng nhiều Logistic Regression, cho chúng hợp tác với nhau.



Mỗi Neuron = 1 Logistic Regression

Hệ phương trình – vai trò của Activation Function

Nhắc lại: 1 Logistic Regression

$\hat{y} = \sigma(w_1x_1 + w_2x_2 + b)$ → học được: 1 đường thẳng phân tách duy nhất

Hệ phương trình: n neurons song song

$a_1 = \sigma(w_{11}x_1 + w_{12}x_2 + b_1)$ ← neuron 1 học ranh giới H_1

$a_2 = \sigma(w_{21}x_1 + w_{22}x_2 + b_2)$ ← neuron 2 học ranh giới H_2

$a_3 = \sigma(w_{31}x_1 + w_{32}x_2 + b_3)$ ← neuron 3 học ranh giới H_3

Dạng ma trận: $a[1] = \sigma(W[1] \cdot x + b[1])$

3 neurons = 3 "chuyên gia"

- H_1 : tách phần trên / dưới
- H_2 : tách phần trái / phải
- H_3 : tách góc còn lại
- → Kết hợp: tách được XOR!

Tại sao cần Activation Function? (2 vai trò)

 Vai trò 1 – Tạo phi tuyến:

Không có σ : $W[2](W[1]x+b[1])+b[2] = W'x+b'$ → 100 layers
= 1 LR

 Vai trò 2 – On/Off (kiểm soát miền giá trị):

$z \in (-\infty, +\infty) \rightarrow \sigma$ quyết định neuron "lên tiếng" hay "im lặng"

Ứng Dụng & Cột Mốc Lịch Sử

Neural Networks qua các thời đại



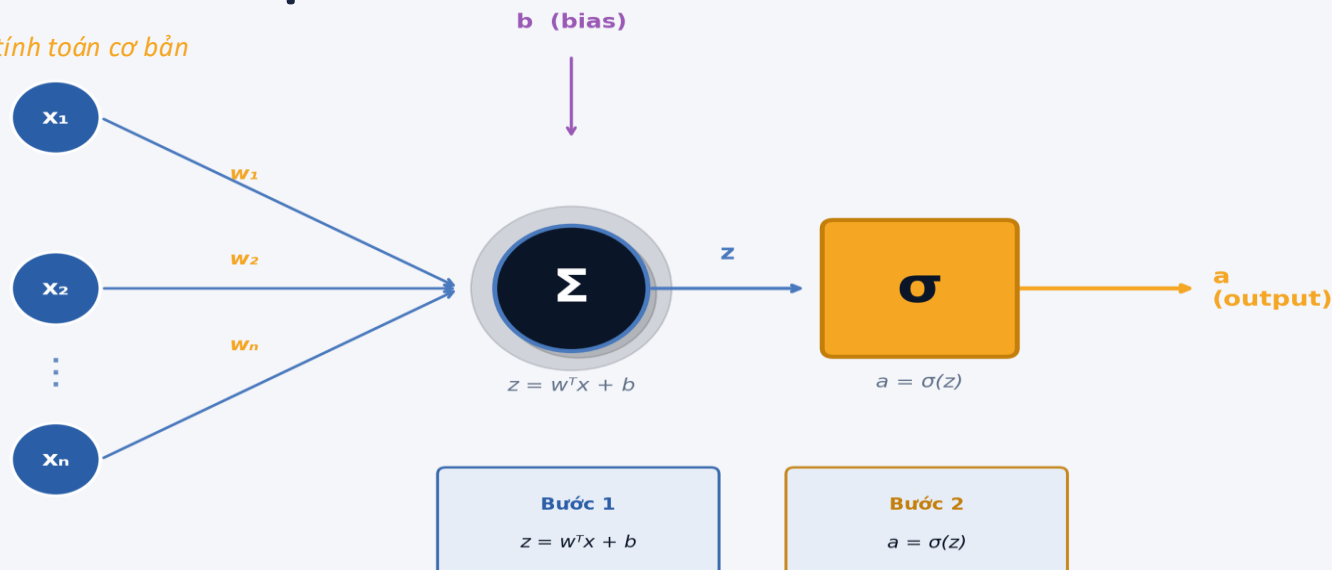
SECTION B

Cấu Trúc Mạng Neural

Neuron · MLP · Forward Pass · Activation · Ma trận

Neuron Nhân Tạo

Đơn vị tính toán cơ bản



Bước 1 – Linear Combination

- $z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b = w^T x + b$
- Kết hợp tuyến tính có trọng số tất cả inputs

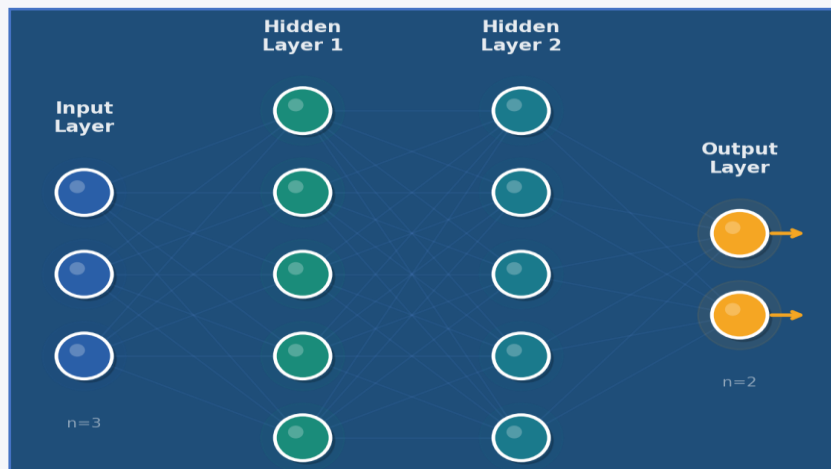
Bước 2 – Activation $\sigma(z)$

- $a = \sigma(z) \rightarrow$ đưa vào phi tuyến tính
- Kiểm soát miền giá trị: quyết định neuron lên tiếng hay im lặng

Kiến Trúc MLP & Từ Shallow đến Deep NN

Ký hiệu chuẩn · Universal Approximation · Hierarchical Learning

Ký hiệu chuẩn



Ký hiệu	Ý nghĩa
L	Số lớp
$N[l]$	Số neurons lớp l
$W[l]$	Ma trận weight lớp l
$B[l]$	Vector bias lớp l
$a[l]$	Activation lớp l
$a[0] = x$	Input = activation lớp 0

1. 1 Neuron (LR)

$$\hat{y} = \sigma(w^T x + b)$$

1 đường thẳng
Linearly separable

2. Shallow NN (1 hidden)

$$\begin{aligned} a[1] &= \sigma(W[1]x + b[1]) \\ \hat{y} &= \sigma(W[2]^T a[1] + b[2]) \end{aligned}$$

Ranh giới cong
Universal Approximation¹

3. Deep NN (nhiều tầng)

$$x \rightarrow a[1] \rightarrow a[2] \rightarrow \dots \rightarrow \hat{y}$$

Biểu diễn phân cấp
Hierarchical learning

¹ Universal Approximation Theorem (Cybenko, 1989): 1 hidden layer với đủ neurons xấp xỉ được bất kỳ hàm liên tục nào.

Forward Pass

Tính $\hat{y}$ từ x – chưa học gì, chỉ tính toán

Công thức tổng quát (layer l)

$$\begin{aligned} z[l] &= W[l] \cdot a[l-1] + b[l] \\ a[l] &= \sigma(z[l]) \end{aligned}$$

Ví dụ số – Mạng [2, 2, 1]

$$\begin{aligned} x &= [1, 2]^T \rightarrow \text{tính } z[1], a[1] \rightarrow \\ &\text{tính } z[2] \rightarrow \hat{y} \end{aligned}$$

1 **Linear:** $z[1] = W[1] \cdot x + b[1]$

Ma trận $(n[1] \times 2) \cdot (2 \times 1) = (n[1] \times 1)$

2 **Activation:** $a[1] = \sigma(z[1])$

σ áp dụng element-wise lên từng phần tử

3 **Linear:** $z[2] = W[2] \cdot a[1] + b[2]$

Ma trận $(1 \times n[1]) \cdot (n[1] \times 1) = (1 \times 1)$

4 **Output:** $\hat{y} = \sigma(z[2])$

Forward pass = luồng dữ liệu từ input $\rightarrow$ output. Lưu cache $(z[l], a[l])$ để dùng lại trong Backprop.



Activation Functions – Phần 1

Tổng quan các hàm kích hoạt phổ biến

Sigmoid

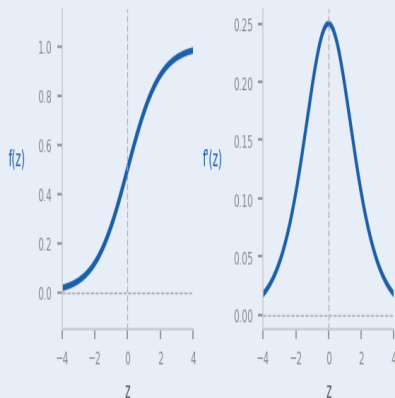
$$\sigma(z) = 1/(1+e^{-z})$$

Range: (0, 1)

Diễn giải như xác suất

⚠ Vanishing gradient khi $|z|$ lớn

📌 **Output layer: binary classification**



Tanh

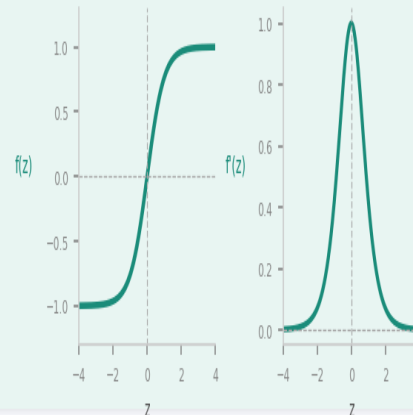
$$\tanh(z) = (e^z - e^{-z}) / (e^z + e^{-z})$$

Range: (-1, 1)

Zero-centered, tốt hơn Sigmoid

⚠ Vẫn có vanishing gradient

📌 **Hidden layers (mạng cũ)**



ReLU ★

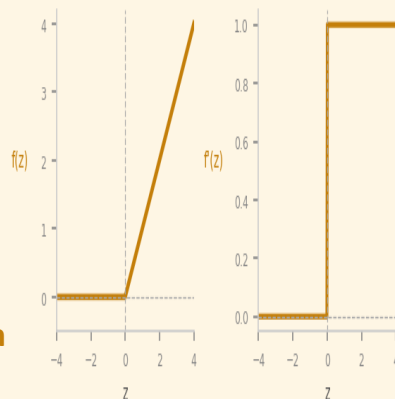
$$\text{ReLU}(z) = \max(0, z)$$

Range: $[0, +\infty)$

✅ Nhanh, ít vanishing gradient

⚠ Dying ReLU khi $z < 0$ mãi

📌 **Hidden layers – mặc định hiện nay**



Softmax

$$\sigma(z_i) = e^{z_i} / \sum e^{z_j}$$

Range: (0,1), tổng = 1

Phân phối xác suất over classes

Chỉ dùng ở output layer

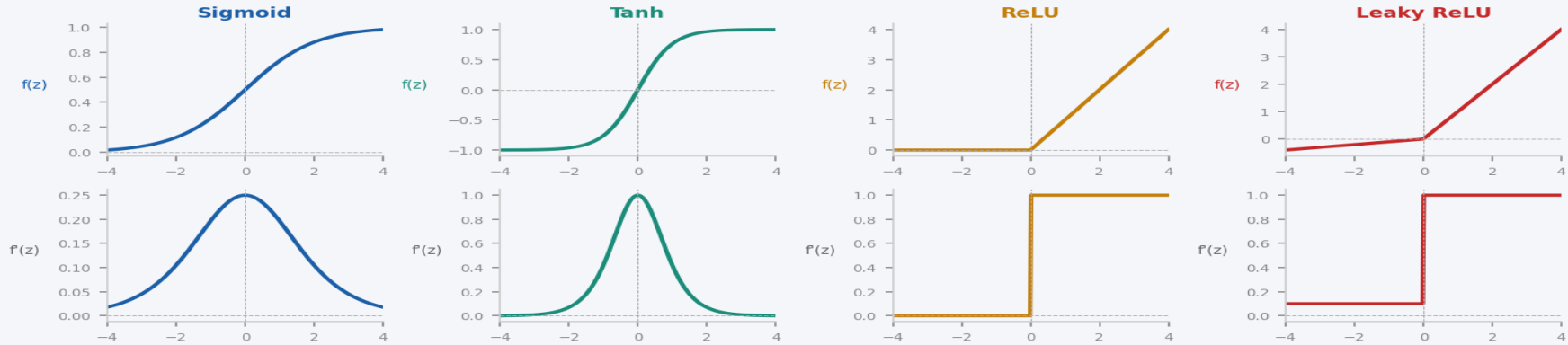
📌 **Output layer: multi-class classification**



FPT SCHOOL OF BUSINESS
& TECHNOLOGY

Activation Functions – Phần 2

Đạo hàm · Bảng so sánh · Cơ chế On/Off



Cơ chế On/Off – Minh họa ReLU

$z = -2.3$ → ReLU → 0

Neuron "im lặng": không đóng góp cho output

$z = +1.7$ → ReLU → 1.7

Neuron "lên tiếng": truyền tín hiệu nguyên giá trị

$z = -0.1$ → ReLU → 0

Neuron "im lặng": bị tắt dùm gần 0

→ Mỗi input kích hoạt một tập con khác nhau của neurons → mạng học được các "chuyên gia" cho từng vùng dữ liệu.



FPT SCHOOL OF BUSINESS
& TECHNOLOGY

Kích Thước Ma Trận

Dimension check – kỹ năng debug số 1

Tensor	Kích thước	Ghi chú
$W[l]$	$(n[l], n[l-1])$	Số hàng = neurons lớp l , số cột = neurons lớp trước
$b[l]$	$(n[l], 1)$	Một bias cho mỗi neuron
$z[l], a[l]$	$(n[l], 1)$	Với 1 mẫu duy nhất
$Z[l], A[l]$	$(n[l], m)$	m = batch size – vectorized implementation

Ví dụ: Mạng [2, 4, 3, 1]

Layer	$W[l]$	$b[l]$
$l=1$ (4 neurons)	(4, 2)	(4, 1)
$l=2$ (3 neurons)	(3, 4)	(3, 1)
$l=3$ (1 neuron)	(1, 3)	(1, 1)

💡 Quy tắc nhớ:
 $W[l] = (n[l], n[l-1])$

Kiểm tra shape sau mỗi phép nhân ma trận khi debug!

Tip: In ra shape của từng tensor trong forward pass để xác nhận đúng dimension trước khi train.

SECTION C

Hàm Mất Mát & Tối Ưu

Loss Functions · Gradient Descent · Optimizers

Hàm Mất Mát (Loss Functions)

Đo "mức độ sai" – mục tiêu là minimize L

MSE – Regression

$$L = (1/m) \sum (y - \hat{y})^2$$

- Penalize sai số bình phương
- Sai lớn bị phạt nặng hơn sai nhỏ
- Nhạy cảm với outliers

BCE – Binary Classification

$$L = -(1/m) \sum [y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$$

- Nguồn gốc: Maximum Likelihood Estimation
- Phạt nặng khi dự đoán ngược chiều
- Kết hợp với Sigmoid ở output

CCE – Multi-class Classification

$$L = -(1/m) \sum_i \sum_k y_k \cdot \log(\hat{y}_k)$$

- Tổng quát hóa BCE cho K classes
- Kết hợp với Softmax ở output layer
- One-hot encoding cho y

Loss = thước đo hiệu suất mô hình trên tập huấn luyện. Training = quá trình tìm W, b để minimize Loss.



Gradient Descent

Tối ưu hóa – đi ngược chiều gradient để giảm Loss

Công thức cập nhật

$$\theta \leftarrow \theta - \eta \cdot \nabla_{\theta} L$$

θ Tham số

Weights W và biases b cần cập nhật

η Learning rate

"Bước đi" – dài hay ngắn?

$\nabla_{\theta} L$

Hướng dốc nhất của Loss surface

Ảnh hưởng của Learning Rate η

η quá nhỏ: Hội tụ rất chậm, tốn nhiều epoch, có thể bị kẹt local minimum

η quá lớn: Dao động, không hội tụ, loss có thể tăng, weights phát nổ

η vừa phải : Hội tụ nhanh và ổn định – thường trong khoảng [0.001, 0.1]

Các Biến Thể Gradient Descent

Batch · SGD · Mini-batch

Loại	Batch size	Ưu điểm	Nhược điểm
Batch GD	Toàn bộ m	Gradient chính xác, ổn định	Chậm, tốn RAM khi m lớn
Stochastic GD	1 mẫu	Nhanh, online learning	Noise lớn, không ổn định
Mini-batch GD ★	32–256	✅ Cân bằng tốt, GPU friendly	Cần chọn batch size phù hợp

Mini-batch GD = chuẩn thực tế. Batch size thường là lũy thừa của 2 (32, 64, 128, 256) để tận dụng GPU parallelism.

Loss curve theo epoch (minh họa)

- Batch GD: đường mượt, giảm đều đặn
- SGD: zigzag nhiều, noisy nhưng hội tụ được
- Mini-batch GD: zigzag vừa phải – tốt nhất trong thực tế



FPT SCHOOL OF BUSINESS
& TECHNOLOGY

Optimizers Nâng Cao

Momentum · RMSprop · Adam · LR Schedule

Momentum

$$v \leftarrow \beta v + (1-\beta)\nabla L$$
$$\theta \leftarrow \theta - \eta \cdot v$$

Tích lũy "quán tính" theo hướng gradient. Vượt qua saddle points nhanh hơn.

RMSprop

$$s \leftarrow \beta s + (1-\beta)(\nabla L)^2$$
$$\theta \leftarrow \theta - \eta \cdot \nabla L / \sqrt{s + \epsilon}$$

Adaptive learning rate theo từng tham số. Tốt cho dữ liệu non-stationary.

Adam ★ (mặc định)

Momentum + RMSprop
AMSGrad, AdamW là các biến thể

Kết hợp ưu điểm của cả hai. Lựa chọn mặc định cho hầu hết bài toán.

LR Schedule

Step decay, Cosine annealing
Warm-up + Decay

Giảm η theo epoch. Bắt đầu học nhanh, tinh chỉnh chậm ở cuối training.

Bài thực hành dùng Adam. Hiểu Gradient Descent cơ bản là đủ nền tảng – Optimizers là cải tiến kỹ thuật thêm.



SECTION D

Backpropagation

Chain Rule · 4 Phương Trình Cốt Lõi · Vanishing Gradient



Bài Toán: Làm Sao Tính Gradient?

Tại sao cần Backpropagation?

? Câu hỏi trung tâm

Để cập nhật $W[l]$, ta cần $\partial L / \partial W[l]$ – nhưng $W[l]$ không ảnh hưởng trực tiếp đến L !

Chuỗi phụ thuộc (Dependency Chain)

$$W[1] \rightarrow z[1] \rightarrow a[1] \rightarrow \dots \rightarrow a[L] \rightarrow L$$

✓ Giải pháp: Chain Rule

$$\partial L / \partial W[1] = \partial L / \partial a[1] \cdot \partial a[1] / \partial z[1] \cdot \partial z[1] / \partial W[1]$$

Chain rule = nhân chuỗi đạo hàm dọc theo đường từ $W[l]$ đến L

Analogy: Chain rule = "dây chuyền truyền trách nhiệm gây lỗi" ngược từ output về input.

BP tính gradient cho TẤT CẢ layers chỉ trong 1 lần backward pass – hiệu quả hơn tính từng layer riêng lẻ.

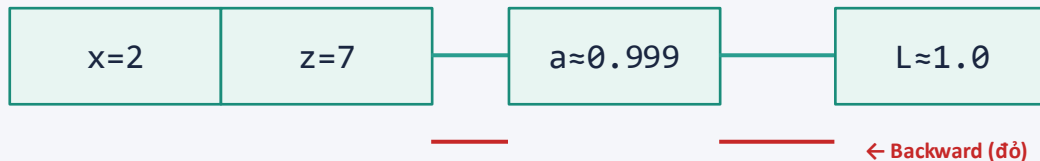
Chain Rule – Ví Dụ Số Cụ Thể

Computation graph: Forward & Backward

Setup

- $x = 2, w = 3, b = 1$
- $z = w \cdot x + b = 7$
- $a = \sigma(7) \approx 0.999$
- $L = (a - y)^2, y = 0$

Forward (xanh) →



Tính Gradient – Backward Pass

$$\partial L / \partial a = 2(a - y) = 2(0.999 - 0) \approx 2.0$$

$$\partial a / \partial z = \sigma'(z) = \sigma(z) \cdot (1 - \sigma(z)) \approx 0.001 \quad \leftarrow \text{đạo hàm Sigmoid}$$

$$\partial z / \partial w = x = 2$$

$$\partial z / \partial b = 1$$

$$\therefore \partial L / \partial w = \partial L / \partial a \cdot \partial a / \partial z \cdot \partial z / \partial w = 2.0 \times 0.001 \times 2 \approx 0.004$$

$$\therefore \partial L / \partial b = \partial L / \partial a \cdot \partial a / \partial z \cdot \partial z / \partial b = 2.0 \times 0.001 \times 1 \approx 0.002$$

Backprop – 4 Phương Trình Cốt Lõi

Error signal $\delta[l] = \partial L / \partial z[l]$

Định nghĩa: $\delta[1] = \partial L / \partial z[1]$ (error signal tại layer 1)

#	Công thức	Ý nghĩa
BP1	$\delta[L] = \nabla_a L \odot \sigma'(z[L])$	Error tại output layer
BP2	$\delta[l] = (W[l+1])^T \cdot \delta[l+1] \odot \sigma'(z[l])$	Truyền ngược error về layer 1
BP3	$\partial L / \partial W[l] = \delta[l] \cdot (a[l-1])^T$	Gradient của ma trận weight
BP4	$\partial L / \partial b[l] = \delta[l]$	Gradient của vector bias

$\odot$ = element-wise multiplication (Hadamard product)

BP1 & BP2 – Truyền ngược error

- BP1: khởi động từ output layer
- BP2: từ layer L-1 ngược về layer 1
- Mỗi bước nhân thêm $(W[l+1])^T$ và $\sigma'(z[l])$

BP3 & BP4 – Tính gradient

- BP3: outer product $\delta[l] \cdot (a[l-1])^T \rightarrow$ gradient W
- BP4: $\delta[l]$ trực tiếp là gradient b
- Update: $W \leftarrow W - \eta \cdot \partial L / \partial W$, $b \leftarrow b - \eta \cdot \partial L / \partial b$



Backprop – Minh Họa & Pseudocode

Forward cache → Backward gradient → Update

```
# FORWARD PASS (lưu cache)
for l in range(1, L+1):
    z[l] = W[l] @ a[l-1] + b[l]
    a[l] = σ(z[l])    # lưu z[l], a[l]

# BACKWARD PASS
δ[L] = grad_loss(a[L], y) * σ'(z[L])
for l in range(L-1, 0, -1):
    δ[l] = W[l+1].T @ δ[l+1] * σ'(z[l])
    dW[l] = δ[l] @ a[l-1].T / m
    db[l] = δ[l].mean(axis=1)

# UPDATE
for l in range(1, L+1):
    W[l] -= η * dW[l]
    b[l] -= η * db[l]
```

Điểm mấu chốt

- ▶ **Cache $z[l]$, $a[l]$**

Forward pass lưu lại để Backward dùng lại $\sigma'(z[l])$

- ▶ **1 Forward + 1 Backward**

Tính gradient cho tất cả tham số chỉ trong 2 passes

- ▶ **BP2: chuỗi nhân**

$$\delta[l] = (W[l+1])^T \cdot \delta[l+1] \cdot \sigma'(z[l])$$

- ▶ **Divide by m**

Trung bình gradient theo batch size



Vanishing & Exploding Gradient

Thách thức khi huấn luyện mạng sâu

⚠ Vanishing Gradient

- Sigmoid/Tanh: $\sigma'(z) \leq 0.25$
- Nhân qua L layers: $(0.25)^L \rightarrow 0$
- Gradient ≈ 0 tại các lớp đầu
- $\rightarrow$ Lớp đầu không học được gì

-  Fix:
 - Dùng ReLU thay Sigmoid/Tanh
 - Batch Normalization
 - Residual connections (ResNet)
 - Khởi tạo He / Xavier

💣 Exploding Gradient

- Gradient tăng theo cấp số nhân
- Weights $\rightarrow \infty$, Loss = NaN
- Xảy ra khi weights lớn + nhiều layers
- $\rightarrow$ Training sụp đổ hoàn toàn

-  Fix:
 - Gradient Clipping
 - Khởi tạo weights nhỏ (He, Xavier)
 - Batch Normalization
 - Giảm learning rate

 Lý do ResNet ra đời (He et al., 2016): Skip connections $y = F(x) + x$ cho phép gradient đi thẳng qua $\rightarrow$ giải quyết vanishing gradient trong mạng rất sâu (100+ layers).

SECTION E

Thực Hành – Demo

NumPy Scratch · Keras · Visualization



Kế Hoạch Demo

Dataset: *make_moons (sklearn)* – Binary Classification

1

NumPy Scratch

Tự implement forward pass, loss, backprop, update từ công thức. Mục tiêu: mỗi dòng code khớp với 1 phương trình toán.

2

Keras / PyTorch

Cùng bài toán, dùng high-level API. So sánh số dòng code và kết quả accuracy.

3

Visualize & Experiment

Plot decision boundary, loss curve. Thử thay đổi: số hidden units, learning rate, activation function.

Mục tiêu: thấy rõ mối liên hệ trực tiếp giữa công thức toán và code Python. Sau đó dùng framework để làm việc thực tế.



Demo: Forward Pass & Loss (NumPy)

Khởi tạo tham số → Forward → Tính Loss

```
import numpy as np
from sklearn.datasets import make_moons

X, y = make_moons(n_samples=300, noise=0.2, random_state=42)
y = y.reshape(1, -1)    # shape: (1, 300)
X = X.T                  # shape: (2, 300)

def init_params(layer_dims):
    params = {}
    for l in range(1, len(layer_dims)):
        params[f'W{l}'] = np.random.randn(layer_dims[l], layer_dims[l-1]) * 0.01
        params[f'b{l}'] = np.zeros((layer_dims[l], 1))
    return params

def forward(X, params):
    Z1 = params['W1'] @ X + params['b1']    # (16, 2)@(2, m)
    A1 = np.tanh(Z1)                        # hidden: tanh
    Z2 = params['W2'] @ A1 + params['b2']   # (1, 16)@(16, m)
    A2 = 1 / (1 + np.exp(-Z2))              # output: sigmoid
    return A2, (Z1, A1, Z2, A2)
```

→ In shape: A1.shape=(16,300), A2.shape=(1,300) để kiểm tra trước khi train



Demo: Backprop & Update (NumPy)

4 phương trình BP trong code Python

```
def compute_loss(A2, y):
    m = y.shape[1]
    return -np.mean(y*np.log(A2+1e-8) + (1-y)*np.log(1-A2+1e-8))

def backward(X, y, params, cache):
    m = X.shape[1]
    Z1, A1, Z2, A2 = cache

    dZ2 = A2 - y                                # BP1: BCE+Sigmoid → đơn giản!
    dW2 = (1/m) * dZ2 @ A1.T                     # BP3
    db2 = (1/m) * np.sum(dZ2, axis=1, keepdims=True) # BP4

    dZ1 = params['W2'].T @ dZ2 * (1-A1**2)      # BP2: đạo hàm tanh
    dW1 = (1/m) * dZ1 @ X.T                     # BP3
    db1 = (1/m) * np.sum(dZ1, axis=1, keepdims=True) # BP4

    return {'dW1':dW1,'db1':db1,'dW2':dW2,'db2':db2}

def update(params, grads, lr=0.1):
    for l in [1, 2]:
        params[f'W{l}'] -= lr * grads[f'dW{l}']
        params[f'b{l}'] -= lr * grads[f'db{l}']
    return params
```

Demo: Training Loop & Kết Quả

Loss curve · Decision boundary · Accuracy ~95%

```
params = init_params([2, 16, 1])
losses = []

for epoch in range(3000):
    A2, cache = forward(X, params)
    loss = compute_loss(A2, y)
    grads = backward(X, y, params, cache)
    params = update(params, grads, lr=0.1)
    if epoch % 100 == 0:
        losses.append(loss)
```

```
# Keras - cùng kết quả, ít code hơn
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
model = Sequential([
    Dense(16, activation='relu', input_shape=(2,)),
    Dropout(0.3),
    Dense(1, activation='sigmoid')
])
model.compile(optimizer='adam', loss='binary_crossentropy',
              metrics=['accuracy'])
model.fit(X.T, y.T, epochs=100, batch_size=32)
```

Kết quả mong đợi

- Loss epoch 0: ~0.693 (random)
- Loss epoch 1000: ~0.12
- Loss epoch 3000: ~0.05
- Accuracy: ~95% trên make_moons
- Decision boundary: đường cong

NumPy vs Keras

- NumPy scratch: ~60 dòng
- → Hiểu sâu từng gradient
- Keras: ~10 dòng
- → Production, dễ mở rộng



FPT SCHOOL OF BUSINESS
& TECHNOLOGY

SECTION F

Thực Tế & Mở Rộng

Overfitting · Khởi tạo · Recap · Cầu nối



Vấn Đề Thực Tế: Overfitting

Biểu hiện và cách giải quyết

⚠ Biểu hiện Overfitting

- Train loss giảm ↓ nhưng Val loss tăng ↑
- Decision boundary quá phức tạp
- Mô hình học cả noise trong train data
- → Không generalize tốt trên test data

Kỹ thuật	Cơ chế	Khi dùng
L2 Regularization	Penalize $ W ^2 \rightarrow$ weights nhỏ hơn	Luôn thử trước
Dropout	Tắt ngẫu nhiên p% neurons	Deep network
Early Stopping	Dừng khi val loss tăng	Nhanh, đơn giản
Data Augmentation	Tăng dữ liệu giả	CV, NLP

```
# Keras - Regularization
from tensorflow.keras.regularizers import l2
model.add(Dense(64, activation='relu', kernel_regularizer=l2(0.01)))
model.add(Dropout(0.3)) # tắt 30% neurons ngẫu nhiên trong training
```

Quy trình thực tế: (1) Overfit trước – chứng minh mạng đủ capacity. (2) Sau đó thêm regularization để giảm overfitting. Đừng regularize quá sớm!



Khởi Tạo Tham Số & Batch Normalization

Giảm vanishing/exploding gradient từ đầu

Weight Initialization

- Zero init: ❌ tất cả neurons giống nhau
- Random nhỏ: tránh symmetry breaking
- Xavier/Glorot (Sigmoid, Tanh):
 - $W \sim N(0, 2/(n[l-1]+n[l]))$
- He initialization (ReLU):
 - $W \sim N(0, 2/n[l-1])$
- → Gradient ổn định từ epoch đầu tiên

Batch Normalization

- Normalize $z[l]$ theo từng mini-batch:
- $\mu = \text{mean}(z[l])$
- $\sigma^2 = \text{var}(z[l])$
- $z_{\text{norm}} = (z[l] - \mu) / \sqrt{\sigma^2 + \epsilon}$
- $z_{\text{out}} = \gamma \cdot z_{\text{norm}} + \beta$
- ☒ Giảm internal covariate shift
- ☒ Cho phép learning rate lớn hơn
- ☒ Có tác dụng regularization nhẹ

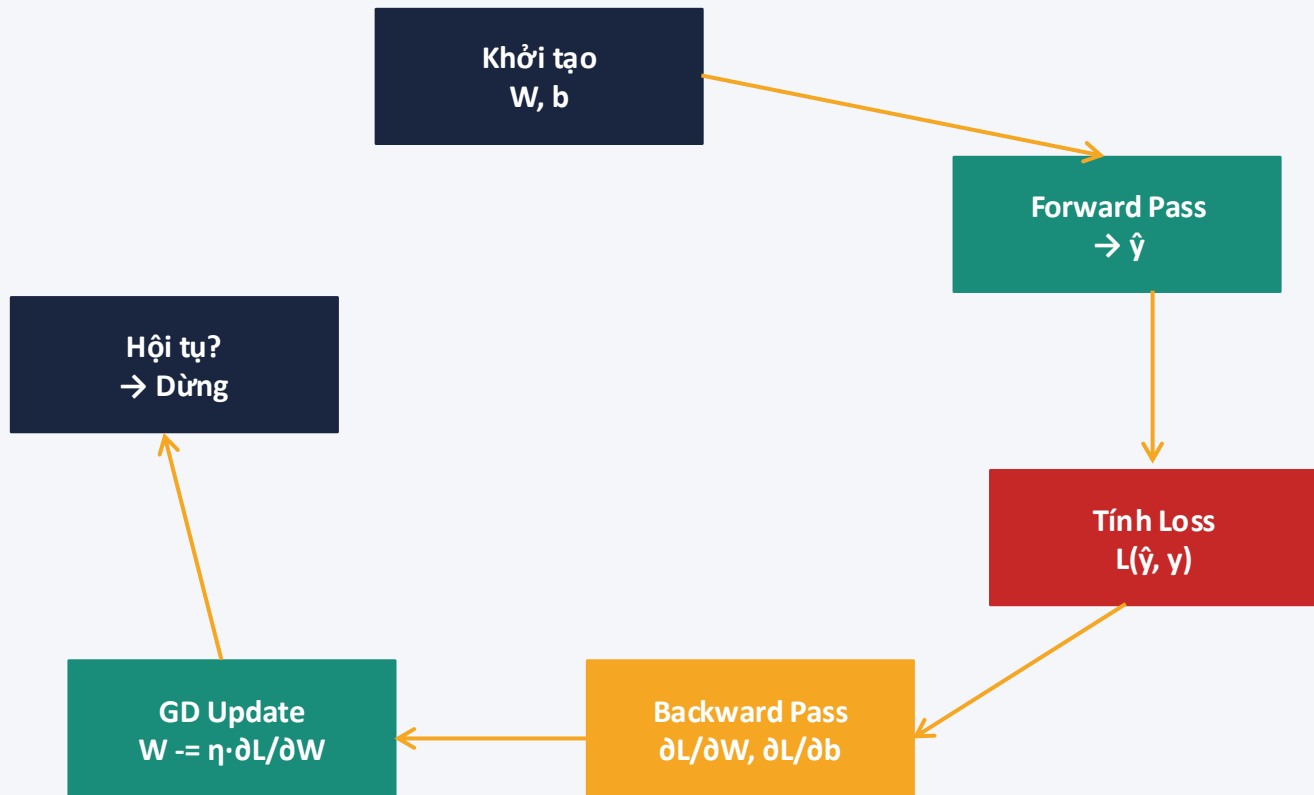
```
# Keras - He init + BatchNorm
from tensorflow.keras.layers import BatchNormalization
model.add(Dense(64, activation='relu', kernel_initializer='he_normal'))
model.add(BatchNormalization()) # thêm sau Dense, trước Activation
```

Recap: Toàn Bộ Pipeline Huấn Luyện

Từ khởi tạo đến hội tụ

✓ Checklist

- ✓ Kiểm tra shape ma trận
- ✓ Loss giảm sau epoch 1
- ✓ Không có NaN/Inf
- ✓ Gradient check OK
- ✓ Overfit trước
- ✓ Rồi regularize



Cầu Nối: Tiếp Theo Sau MLP

Backprop + GD là nền tảng của TẤT CẢ kiến trúc sau



CNN

Convolutional NN

Chia sẻ trọng số theo không gian. Phù hợp cho ảnh, video, tín hiệu 2D.



RNN / LSTM / GRU

Recurrent NN

Xử lý chuỗi thời gian, văn bản. Hidden state = "bộ nhớ" qua các timesteps.



Transformer

Attention mechanism

NLP hiện đại, ChatGPT, BERT. Self-attention thay thế recurrence.



GNN

Graph NN

Dữ liệu dạng đồ thị, mạng xã hội, phân tử hóa học.

💡 Dù là CNN, RNN hay Transformer – tất cả đều dùng *Backpropagation + Gradient Descent* để học. MLP là nền tảng bắt buộc phải nắm vững.



Câu Hỏi Ôn Tập

Kiểm tra hiểu biết cốt lõi

1

Tại sao 1 Logistic Regression không giải được bài toán XOR?

2

Điều gì xảy ra nếu bỏ activation function σ ?

3

Activation function kiểm soát miền giá trị output như thế nào? Cho ví dụ với ReLU.

4

Giải thích BP2: $\delta[l] = (W[l+1])^T \cdot \delta[l+1] \odot \sigma'(z[l])$. Mỗi thành phần có nghĩa gì?

5

Vanishing gradient xảy ra khi nào? Nêu 2 nguyên nhân và 2 cách khắc phục.



Tài Liệu Tham Khảo

Đọc thêm & công cụ thực hành

Loại	Nguồn	Nội dung
 Sách	Deep Learning – Goodfellow et al.	Chương 6–8: MLP, Backprop, Regularization
 Video	3Blue1Brown – Neural Networks series	Trực quan hóa xuất sắc về backprop
 Interactive	playground.tensorflow.org	Thử nghiệm NN trực tiếp trên trình duyệt
 Paper	Rumelhart et al. (1986) – Nature	Learning representations by back-propagating errors
 Notebook	Bài thực hành – link Jupyter Notebook	NumPy scratch + Keras so sánh